

NEURAL NETWORKS

Experiments 2–10

Each experiment includes the aim from the uploaded notebook pages, followed by the corresponding program discussed in this chat.

Experiment 6 is Python as written in its notebook aim. The other programs are MATLAB.

Toolbox note: Experiment 4 uses `fitnet/train`, and Experiment 8 uses `selforgmap/train`; these need the relevant MATLAB neural-network toolbox.

Experiment 2 — McCulloch-Pitts Neuron

Aim: To study the McCulloch-Pitts (MP) neuron model and implement AND, OR, NOT and XOR logic gates using MATLAB.

Program:

```
clc; clear; close all;
X = [0 0; 0 1; 1 0; 1 1];
AND = double(sum(X,2) >= 2);
OR = double(sum(X,2) >= 1);
N = [0; 1]; NOT = double(N < 1);
ORh = double(sum(X,2) >= 1);
NANDh = double(sum(X,2) < 2);
XOR = double(ORh + NANDh >= 2);
disp('AND Gate:'); disp([X AND]);
disp('OR Gate:'); disp([X OR]);
disp('NOT Gate:'); disp([N NOT]);
disp('XOR Gate:'); disp([X XOR]);
```

Experiment 3 — Artificial Neuron Output

Aim: To implement a simple artificial neuron using the weighted sum model and activation functions, and to calculate and plot the neuron's output for different input values.

Program:

```
clc; clear; close all;
x = -5:0.1:5; w = 1; b = 0;
net = w*x + b;
step = double(net >= 0);
sigmoid = 1./(1 + exp(-net));
figure;
plot(x, sigmoid, 'LineWidth', 2);
grid on; xlabel('Input'); ylabel('Neuron Output');
title('Artificial Neuron Output - Sigmoid');
```

Experiment 4 — Custom Artificial Neural Network

Aim: To define a custom Artificial Neural Network (ANN), train it using sample data, and calculate the network output using MATLAB.

Program:

```
clc; clear; close all;
x = 0:0.1:1;
t = x.^2;
net = fitnet(5);
net = train(net, x, t);
y = net(x);
disp('Network Output:'); disp(y);
figure; plot(x, t, 'o', x, y, '-');
grid on; legend('Target', 'Network Output');
title('Custom ANN Output');
```

Experiment 5 — Perceptron Learning Algorithm

Aim: To implement the Perceptron learning algorithm for solving linearly separable classification problems.

Program:

```
clc; clear;
eta = 1; X = [0 0; 0 1; 1 0; 1 1];
% AND gate
T = [0; 0; 0; 1]; w = [0 0]; b = 0;
for epoch = 1:10
    for i = 1:4
        y = double(w*X(i,:) + b >= 0); e = T(i)-y;
        w = w + eta*e*X(i,:); b = b + eta*e;
    end
end
disp('AND Perceptron Output:'); disp(double(X*w' + b >= 0));
% OR gate
T = [0; 1; 1; 1]; w = [0 0]; b = 0;
for epoch = 1:10
    for i = 1:4
        y = double(w*X(i,:) + b >= 0); e = T(i)-y;
        w = w + eta*e*X(i,:); b = b + eta*e;
    end
end
disp('OR Perceptron Output:'); disp(double(X*w' + b >= 0));
% Binary classification using AND-labelled data
T = [0; 0; 0; 1]; w = [0 0]; b = 0;
for epoch = 1:10
    for i = 1:4
        y = double(w*X(i,:) + b >= 0); e = T(i)-y;
        w = w + eta*e*X(i,:); b = b + eta*e;
    end
end
disp('Binary Classification Results:');
for i = 1:4
    y = double(w*X(i,:) + b >= 0);
    fprintf('Input: [%d %d] -> Class %d\n', X(i,1), X(i,2), y);
end
```

Experiment 6 — Binary Pattern Classification (Python)

Aim: To implement the perceptron learning algorithm for binary pattern classification using Python.

Program:

```
import numpy as np
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
T = np.array([0, 0, 0, 1])
w = np.zeros(2); b = 0; lr = 1
for epoch in range(10):
    for i in range(len(X)):
        y = 1 if np.dot(w, X[i]) + b >= 0 else 0
        e = T[i] - y
        w += lr * e * X[i]
        b += lr * e
print('Weights:', w)
print('Bias:', b)
for x in X:
    y = 1 if np.dot(w, x) + b >= 0 else 0
    print(x, '->', y)
```

Experiment 7 — XOR with Backpropagation and Momentum

Aim: To implement the XOR function using a Multilayer Perceptron (MLP) trained with the backpropagation algorithm incorporating a momentum factor.

Program:

```
clc; clear; close all;
X = [0 0; 0 1; 1 0; 1 1]; T = [0; 1; 1; 0];
rng(1); W1 = rand(2,2)-0.5; W2 = rand(1,2)-0.5;
b1 = zeros(2,1); b2 = 0; eta = 0.5; alpha = 0.9; epochs = 10000;
dw1 = zeros(size(W1)); dw2 = zeros(size(W2)); db1 = zeros(size(b1)); db2 = 0;
sig = @(z) 1./(1+exp(-z));
for epoch = 1:epochs
    for i = 1:4
        x = X(i,:); t = T(i); h = sig(W1*x+b1); y = sig(W2*h+b2);
        delta2 = (t-y)*y*(1-y); delta1 = h.*(1-h).*(W2'*delta2);
        nw2 = eta*delta2*h' + alpha*dw2; nw1 = eta*delta1*x' + alpha*dw1;
        nb2 = eta*delta2 + alpha*db2; nb1 = eta*delta1 + alpha*db1;
        W2 = W2+nw2; W1 = W1+nw1; b2 = b2+nb2; b1 = b1+nb1;
        dw2 = nw2; dw1 = nw1; db2 = nb2; db1 = nb1;
    end
end
disp('XOR Results:');
for i = 1:4
    h = sig(W1*X(i,:)'+b1); y = sig(W2*h+b2);
    fprintf('%d XOR %d = %.3f -> %d\n', X(i,1), X(i,2), y, y>=0.5);
end
```

Experiment 8 — SOM Clustering

Aim: To implement clustering of multidimensional data using a Self-Organizing Map (SOM) in MATLAB and visualize the clusters formed by the network.

Program:

```
clc; clear; close all;
X = [1 1.2 0.8 1.1 5 5.2 4.8 5.1; 1 0.8 1.2 0.9 5 4.8 5.2 4.9];
net = selforgmap([2 2]);
net = train(net, X);
Y = net(X); [~, cluster] = max(Y, [], 1);
disp('Cluster assignments:'); disp(cluster);
figure; gscatter(X(1,:), X(2,:), cluster);
xlabel('Feature 1'); ylabel('Feature 2');
title('SOM Clustering'); grid on;
```

Experiment 9 — Discrete Hopfield Network (1110)

Aim: To implement a Discrete Hopfield Network in MATLAB for storing the binary pattern 1110 and recalling the original pattern when the test input contains mistakes in the first and second positions.

Program:

```
clc; clear; close all;
x = [1; 1; 1; -1];
W = x*x'; W(1:5:end) = 0;
disp('Weight Matrix:'); disp(W);
test = [-1; -1; 1; -1];
for k = 1:10
    old = test;
    for i = 1:4
        s = W(i,:)*test;
        if s > 0, test(i) = 1; elseif s < 0, test(i) = -1; end
    end
    if isequal(old,test), break; end
end
recalled = double(test == 1);
disp('Original Pattern:'); disp(double(x == 1));
disp('Noisy Input:'); disp([0 0 1 0]);
disp('Recalled Pattern:'); disp(recalled);
```

Experiment 10 — Hopfield Network for Digits 0–9

Aim: To implement a Discrete Hopfield Network in MATLAB for storing and recalling patterns representing decimal digits (0–9), and to test the network's ability to recover a digit from a noisy or incomplete pattern.

Program:

```
clc; clear; close all;
P = [1 1 1 1 0 1 1 1 1; 0 1 0 1 1 0 0 1 0; 1 1 1 0 1 1 1 1 0; 1 1 0 1 1 1 1 1 1; 1 0 1 1 1 1 0 0 1; 1 1 1 1 1 0 0 1 1; 1 1 1 1 1 0 1 1 1; 1 1 1 1 1 0 1 1 1; 1 1 1 1 1 0 1 1 1];
P(P == 0) = -1; W = zeros(9,9);
for k = 1:10, W = W + P(k,:)'*P(k,:); end
W(1:10:end) = 0;
original = P(1,:); noisy = original; noisy([2 5]) = -noisy([2 5]);
for epoch = 1:20
    old = noisy;
    for i = 1:9
        s = W(i,:)*noisy;
        if s > 0, noisy(i) = 1; elseif s < 0, noisy(i) = -1; end
    end
    if isequal(old,noisy), break; end
end
disp('Original digit 0:'); disp(reshape(original == 1,3,3));
disp('Noisy pattern:'); disp(reshape((original.*[1;-1;1;1;-1;1;1;1;1]) == 1,3,3));
disp('Recalled pattern:'); disp(reshape(noisy == 1,3,3));
figure;
subplot(1,3,1); imagesc(reshape(original == 1,3,3)); title('Original'); axis image;
subplot(1,3,2); imagesc(reshape((original.*[1;-1;1;1;-1;1;1;1;1]) == 1,3,3)); title('Noisy'); axis image;
subplot(1,3,3); imagesc(reshape(noisy == 1,3,3)); title('Recalled'); axis image;
colormap(gray);
```

Note: Hopfield networks storing multiple patterns may recall another pattern or a spurious state; digit recovery is not guaranteed.